Predicting Computational Cost from the Structural Parameter δ: Separating Existence from Discovery in Random 3-SAT

Akihito Sunagawa*

March 2026

Abstract

Whether a solution *exists* and whether an algorithm can *find* it are distinct questions governed by different mechanisms. In random 3-SAT, the first moment method gives $P(\mathrm{SAT}) \approx N_{\mathrm{eff}} \cdot e^{-\delta}$, where $\delta = \alpha n|\ln(7/8)|$ is a reparameterization of the clause density (the exponent in the first moment method). This characterizes existence but says nothing about discovery.

We introduce the computational budget μ (solver operations) and ask: can the conditional success probability $P(\text{found} \mid \mathrm{SAT})$ be multiplicatively decomposed as a function of $\mu/\mu_c(\delta)$?

Experiments with three solver families (CDCL, WalkSAT, random search) on instances with $n = 100$–300 variables and $\alpha = 3.0$–4.0 yield three findings. (1) The median runtime scales as $\mu_c \propto e^{c\delta}$, where the *sensitivity exponent* c is solver-dependent: $c \approx 0.24$ for CDCL, $c \approx 0.21$ for WalkSAT, and $c = 1.0$ for random search (exact), in the floor-free regime $\alpha \geq 3.5$ (fits including $\alpha = 3.0$, where solver startup overhead dominates, give $c = 0.25$ and 0.18 respectively; see Section 4.1). (2) The ratio μ/μ_c is an effective normalizing variable: both solvers exhibit a monotone transition near $\mu/\mu_c = 1$ with $x_0 \approx 0$, though transition shapes differ by 14–15% (model-independent across sigmoid and Weibull fits). (3) The ordering $c_{\mathrm{CDCL}} > c_{\mathrm{WalkSAT}}$ is robust across problem sizes ($n = 100$–300), though both estimates show moderate n-dependence (range ∼18% for CDCL, ∼7% for WalkSAT) when measured at a common clause density range ($\alpha = 3.5$–4.0). A cross-n prediction test (train on $n \leq 200$, predict $n = 300$) achieves factor-of-2 accuracy (RMSE = 0.73 in log space) over a range where μ_c spans three orders of magnitude (Appendix B).

The parameter δ thus governs both existence ($c = 1$, mathematical identity) and discovery ($c < 1$, algorithm-dependent), but with qualitatively different sensitivity. Notably, c measures sensitivity to δ, not efficiency: WalkSAT has the smallest c yet the largest absolute cost, because low sensitivity can arise from ignoring structure rather than exploiting it.

Keywords: random 3-SAT, computational budget, existence–discovery separation, first moment method, solver comparison, sensitivity exponent

1 Introduction

The satisfiability threshold of random k-SAT is among the most studied phenomena in combinatorics and statistical physics [1]. The cavity method and its rigorous counterparts determine, with increasing precision, the critical clause density α_c at which solutions cease to exist [2, 3]. These results answer the *existence* question: for a given clause-to-variable ratio α, does a satisfying assignment exist with high probability?

A complementary question receives far less formal attention: given that a solution *exists*, how much computation is needed to *find* it? Worst-case complexity theory provides upper bounds (e.g., Schöning's $(2(1-1/k))^n$ for k-SAT [4]), and empirical studies document heavy-tailed runtime distributions (RTDs) [5, 6, 8, 9]. Cocco and Monasson [11] analytically derived the

*karesansui@ugentropy.com

computational cost of DPLL as a function of clause density, showing that the typical complexity grows exponentially with an α-dependent rate. RTDs characterize the probability of finding a solution within a given time budget, and have been used for algorithm selection and restart policy design [8]. Yet no framework connects RTDs to the structural parameter δ from the first moment method, nor uses δ to predict how the median runtime scales across clause densities.

We formalize this gap through a decomposition that is trivial as an identity but nontrivial as a quantitative program:

$$P(\text{found}) = \underbrace{P(\text{SAT})}_{\text{existence}} \times \underbrace{P(\text{found} \mid \text{SAT})}_{\text{discovery}}. \tag{1}$$

The left factor is well characterized by the first moment method. The right factor—the *discovery probability*—is what we measure.

Our central finding is that the same structural parameter δ (a reparameterization of the clause density α, equivalent to the exponent in the first moment method) governs both factors, but with different strength:

- **Existence:** $P(\text{SAT}) \approx N_{\text{eff}} \cdot e^{-\delta}$ with $c = 1$ (mathematical identity).
- **Discovery:** The median computational cost scales as $\mu_c \propto e^{c\delta}$ with $c < 1$, where c depends on the solver family.

The sensitivity exponent c measures how strongly the solver's cost responds to increasing structural complexity δ. It is *not* an efficiency measure: CDCL ($c \approx 0.24$) is more sensitive to δ than WalkSAT ($c \approx 0.21$), yet CDCL is 10–50× faster in absolute terms. Sophistication increases sensitivity; it does not reduce it.

2 Framework

2.1 Existence: the first moment baseline

Consider a random 3-SAT instance with n variables and $m = \alpha n$ clauses, each a disjunction of 3 literals chosen uniformly at random. A uniformly random assignment satisfies a single clause with probability $1 - 2^{-k} = 7/8$ for $k = 3$. The expected number of satisfying assignments is

$$\mathbb{E}[\#\text{SAT}] = 2^n \cdot (7/8)^m = \exp\bigl(n \ln 2 - \delta\bigr), \tag{2}$$

where

$$\delta \;\equiv\; \alpha\, n\, |\ln(7/8)| \;=\; m \cdot |\ln(7/8)| \tag{3}$$

is a reparameterization of the clause count m (equivalently, of the clause density α at given n), with $|\ln(7/8)| \approx 0.134$ nats per clause. This is the annealed (replica-symmetric) approximation; the 20% gap between the annealed threshold $\alpha_c^{(1)} \approx 5.19$ and the true threshold $\alpha_c \approx 4.27$ [1] reflects replica symmetry breaking corrections that are well understood but irrelevant to our purpose.

Equation (2) is a mathematical identity within the annealed framework. The parameter δ is *not* a new quantity: it is simply $m \cdot |\ln(7/8)|$, computable from the instance specification and identical to the exponent in the first moment method [1]. What is new is the empirical observation that δ also governs *computational cost*, with a solver-dependent proportionality constant (Section 4.1).

2.2 Discovery: introducing the computational budget

Let μ denote the computational budget available to a solver (measured in solver-specific units: conflicts for CDCL, flips for WalkSAT, random trials for random search). We define the **discovery probability**

$$S_{\text{solve}}(\delta, \mu) \equiv P(\text{found} \mid \text{SAT}) \tag{4}$$

as the probability that the solver finds a satisfying assignment within budget μ, conditioned on the instance being satisfiable. This is the CDF of the runtime distribution (RTD) [8, 9], restricted to satisfiable instances.

Our hypothesis is that S_{solve} admits a **multiplicative separation** through the median runtime $\mu_c(\delta)$:

$$S_{\text{solve}}(\delta, \mu) = f\left(\frac{\mu}{\mu_c(\delta)}\right), \tag{5}$$

where f is a monotone increasing function (a CDF-like transition) and the median runtime scales exponentially:

$$\mu_c(\delta) = A \cdot e^{c\,\delta}. \tag{6}$$

Here c is the **sensitivity exponent**—the rate at which computational cost grows per unit of δ—and A is a unit-dependent intercept (absorbing the absolute speed of the solver implementation).

Why exponential? The exponential form in (6) is not an arbitrary modeling choice. If constraints contribute independently to cost—i.e., $\mu_c(\delta_1+\delta_2) = \mu_c(\delta_1)\cdot[\mu_c(\delta_2)/A]$—then the Cauchy functional equation $g(\delta_1 + \delta_2) = g(\delta_1) \cdot g(\delta_2)$ (with $g = \mu_c/A$, $g(0) = 1$, g continuous and monotone increasing) has a unique solution $g(\delta) = e^{c\delta}$ for some $c > 0$ [10]. In other words, (6) is the *only* continuous scaling law compatible with independent accumulation of constraints. The parameter c is the scale constant of this unique solution, measuring the solver's intrinsic sensitivity to structural complexity. (A formal verification of this characterization theorem in Lean 4 is available in the supplementary repository.)

Scope of the sensitivity exponent. Throughout this paper, c is measured by varying the clause density α at *fixed* problem size n. That is, δ serves as a reparameterization of α (at given n) rather than as a universal coordinate mixing α and n. This distinction matters because the scaling of runtime with n (at fixed α) can differ qualitatively from the scaling with α (at fixed n). In particular, stochastic local search solvers are known to scale polynomially with n at the phase transition [7], which is incompatible with treating $\delta = \alpha n|\ln(7/8)|$ as a single exponential predictor across both axes. Section 5.2 discusses this asymmetry in detail.

Crucially, c is invariant under rescaling of μ: if $\mu \to k\mu$, then $A \to kA$ but the slope c in $\ln \mu_c$ vs. δ is unchanged. This makes c a property of the *algorithm*, not of the implementation.

Three testable hypotheses.

H1 (Full separation): μ_c is independent of α (hence of δ). Different α-values would yield the same runtime distribution.

H2 (μ_c-coupled separation): μ_c depends on δ via Eq. (6), and μ/μ_c collapses the survival curves across different α-values onto a single function f.

H3 (Non-separable): No rescaling of μ can collapse curves from different α-values.

The $c = 1$ baseline. For random search (uniformly sampling assignments and checking), the expected number of trials to find a solution equals $2^n/\#\text{SAT} = e^{\delta}/N_{\text{eff}}$. Hence $\mu_c^{\text{random}} \propto e^{\delta}$, giving $c = 1$ exactly. This is not an approximation but a consequence of the first moment identity (2). Any solver with $c < 1$ is, in a precise sense, experiencing less cost growth per unit of structural complexity than a structureless baseline.

3 Experimental Setup

3.1 Instance generation

We generate random 3-SAT instances with $n \in \{100, 200, 300\}$ variables and clause densities $\alpha \in \{3.0,\ 3.5,\ 3.86,\ 4.0\}$. Values $\alpha > 4.2$ are excluded: the fraction of satisfiable instances drops sharply near the satisfiability threshold $\alpha_c \approx 4.27$, introducing *survivor bias*—instances that are found satisfiable at high α are systematically easier than typical satisfiable instances. For each (n, α) pair, we generate 100–200 instances and retain only those confirmed satisfiable (by an unlimited CDCL run).

3.2 Solvers

Three solver families span a range of algorithmic sophistication:

CDCL (MiniSat via PySAT): Conflict-driven clause learning [12] with restarts. Budget unit: *conflicts* (state transitions in the implication graph). Budget range: 10^2–10^5.

WalkSAT (pure Python, restart): Stochastic local search [13] with noise parameter 0.5 and periodic restarts. Budget unit: *flips* (single-variable reassignments). Budget range: 10^3–10^6.

Random search: Uniformly random assignment, check all clauses, repeat. Budget unit: *trials*. Serves as the $c = 1$ theoretical baseline.

3.3 Measurement protocol

For each instance, we first run an unlimited solve to obtain the true runtime (defining μ_c as the median over instances at each α). We then measure S_{solve} at logarithmically spaced budget values, recording the fraction of SAT instances solved within each budget.

The conditional analysis $P(\text{found} \mid \text{SAT})$ (rather than $P(\text{found})$) is essential: at $\alpha = 4.0$, approximately 8% of instances are unsatisfiable, and including them would conflate the existence threshold with the discovery transition.

4 Results

4.1 $\mu_c \propto e^{c\delta}$: the sensitivity exponent

Figure 1 shows $\ln \mu_c$ versus δ for the three solver families. Linear regression on the four α-values yields:

Solver	c	95% CI	R^2	Note
CDCL (conflicts)	0.25	[0.24, 0.29]	0.99	
WalkSAT (flips)	0.18	[0.17, 0.22]	0.96	
Random (trials)	1.0	—	—	Exact (Eq. 2)

Table 1: Sensitivity exponent c from $\ln \mu_c = c\delta + a$ ($n = 100$, $\alpha = 3.0$–4.0). The 95% CIs are bootstrap percentile intervals (200 instances $\times$ 10 000 resamples). The CIs do not overlap, confirming that the ordering $c_{\text{CDCL}} > c_{\text{WalkSAT}}$ is statistically significant. Note: these estimates include $\alpha = 3.0$, where a floor effect biases c (Section 5); restricting to $\alpha \geq 3.5$ gives $c \approx 0.24$ (CDCL) and $c \approx 0.21$ (WalkSAT).

The slope c is invariant under unit rescaling ($\mu \to k\mu$ shifts the intercept a but not the slope), so the difference between CDCL and WalkSAT is not an artifact of comparing conflicts with flips.

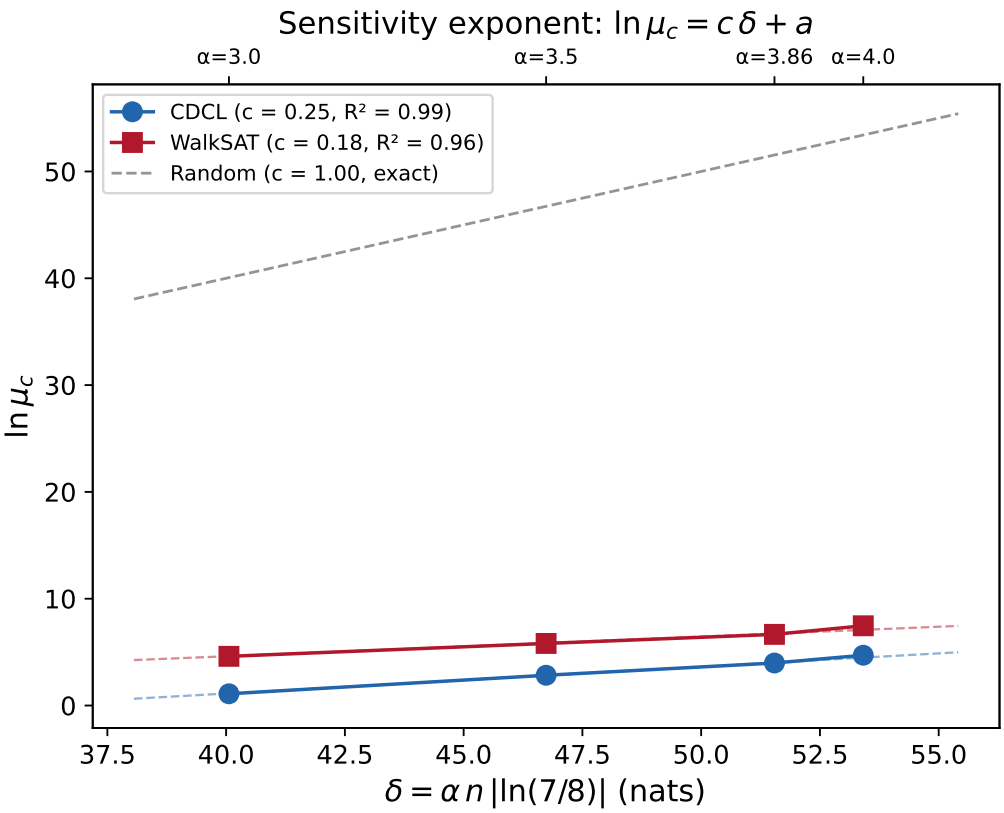

Figure 1: $\ln \mu_c$ versus $\delta = \alpha n |\ln(7/8)|$ for three solver families ($n = 100$, $\alpha = 3.0$–4.0). Dashed lines: linear fits $\ln \mu_c = c\delta + a$. The fit includes $\alpha = 3.0$, where solver startup overhead creates a floor effect (Section 5); restricting to $\alpha \geq 3.5$ gives $c \approx 0.24$ (CDCL) and $c \approx 0.21$ (WalkSAT).

n-dependence of c. Repeating the α-regression at $n = 100$, 200, and 300 (Appendix A) reveals that the estimated c depends on the clause density range. At $\alpha = 3.0$, the median CDCL conflicts are 3–8 across all n—dominated by solver startup overhead rather than exponential scaling. This *floor effect* biases the linear fit: including $\alpha = 3.0$ gives $c \approx 0.19$, while restricting to $\alpha \geq 3.5$ gives $c \approx 0.24$ (see also $c = 0.24$ from the 8-point fine-grained fit in Appendix D).

When measured at a common clause density range ($\alpha = 3.5$–4.0, local slope), both solvers show moderate n-dependence: CDCL gives $c = 0.26$, 0.22, 0.24 and WalkSAT gives $c = 0.21$, 0.21, 0.20 at $n = 100$, 200, 300 (ranges: 18% and 7% respectively). The ordering $c_{\text{CDCL}} > c_{\text{WalkSAT}}$ holds at every n tested, with non-overlapping bootstrap 95% CIs at $n \geq 200$ ($P > 0.99$); at $n = 100$ the two-point floor-free slope has wide CIs and the gap is not significant ($P = 0.76$). The gap (0.02–0.05) is smaller than the full-range estimates suggest. WalkSAT's n-dependence additionally reflects polynomial n-scaling that the linear-in-δ model cannot absorb into the intercept alone (see Section 5.2 and Appendix A).

Interpretation. The ordering $c_{\text{WalkSAT}} < c_{\text{CDCL}} < c_{\text{Random}}$ is initially surprising: the most efficient solver (CDCL) has a *higher* sensitivity to δ than WalkSAT. This reflects two qualitatively different mechanisms for $c < 1$:

- CDCL ($c \approx 0.24$): Clause learning actively exploits the constraint structure, increasing both efficiency *and* sensitivity to that structure.
- WalkSAT ($c \approx 0.21$): Local search largely ignores global constraint structure, making it less responsive to δ—but also less efficient in absolute terms (μ_c is 10–50× larger than CDCL).

Thus c and absolute efficiency $(1/A)$ are independent axes: c measures *how much* the cost grows with δ, while A measures the baseline cost at $\delta = 0$.

4.2 μ/μ_c as a normalizing variable

Figure 2 shows S_{solve} as a function of $\ln(\mu/\mu_c)$ for CDCL and WalkSAT across four α-values.

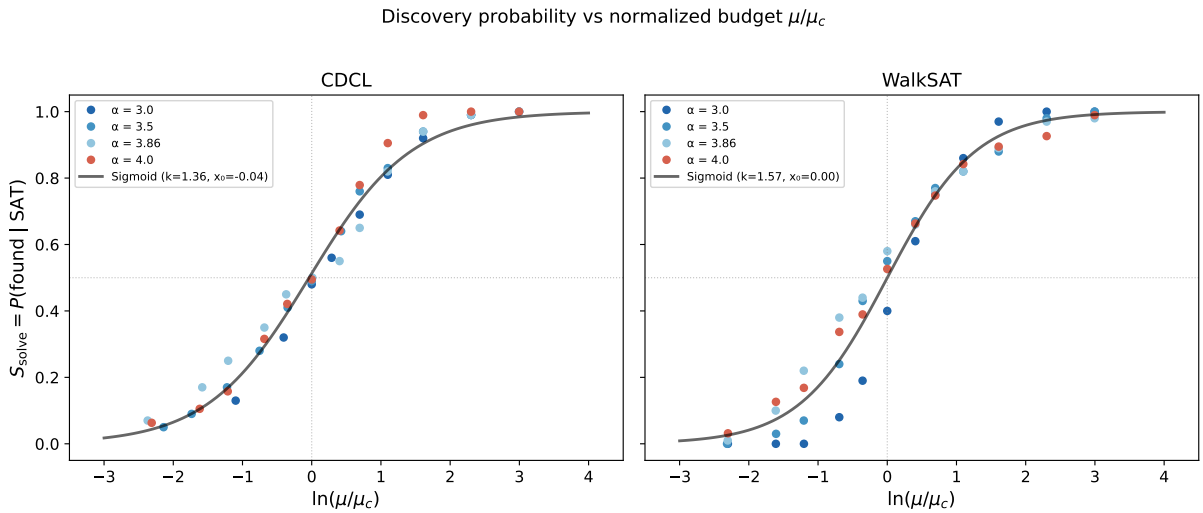

Figure 2: Discovery probability S_{solve} versus $\ln(\mu/\mu_c)$ for CDCL (left) and WalkSAT (right), across four α-values ($n = 100$). Solid curves: sigmoid fits. Both solvers transition near $\mu/\mu_c = 1$ ($x_0 \approx 0$), validating μ_c as a natural budget scale. The steepness k differs by 14% (CDCL: 1.36, WalkSAT: 1.57).

Fitting a logistic (sigmoid) model $S_{\text{solve}} = [1 + \exp(-k(\ln(\mu/\mu_c) - x_0))]^{-1}$ yields:

Fit	k	x_0	R^2	n_{pts}
CDCL only	1.36	−0.04	0.985	45
WalkSAT only	1.57	0.00	0.972	48
Combined	1.46	−0.01	0.976	93

Table 2: Sigmoid fit parameters for the transition curves. Both solvers have $x_0 \approx 0$, confirming that μ_c (the median runtime) naturally marks the 50% transition point. The steepness k differs by 14%.

Two key observations:

1. Both solvers have $x_0 \approx 0$, meaning the 50% success probability occurs near $\mu = \mu_c$. This validates the median runtime as a natural scale for the computational budget.

2. The steepness parameter k differs by 14% (CDCL: 1.36, WalkSAT: 1.57). We do *not* claim a universal scaling function; instead, we report that μ/μ_c is an effective normalizing variable whose transition shape retains solver-specific features.

At $\mu/\mu_c = 1.0$, CDCL achieves $S_{\text{solve}} = 0.48$–0.50 across all four α-values (nearly perfect collapse), while WalkSAT shows $S_{\text{solve}} = 0.40$–0.58 (more scatter). This suggests that conflicts are a more natural unit for μ than flips.

Model independence. To verify that the 14% shape difference is not an artifact of the sigmoid functional form, we also fit a Weibull CDF $S_{\text{solve}} = 1 - \exp(-(\mu/\mu_c)^\beta)$. The Weibull shape parameter shows a comparable 15% difference between solvers (Table 8 in Appendix C). AIC comparison reveals that the best-fit model itself is solver-dependent: Weibull ($\beta = 0.93$) for CDCL, sigmoid for WalkSAT. The shape difference is therefore a genuine property of the runtime distributions, not a fitting artifact.

4.3 Runtime heterogeneity

The Weibull shape parameter β provides insight into runtime distribution structure:

Solver	c	Weibull β	$\Delta\beta$ (vs Random)
Random	1.00	0.87 ± 0.05	baseline
CDCL	0.25	0.93	+0.06
WalkSAT	0.18	1.10	+0.23

Table 3: Weibull shape parameter by solver. $\beta < 1$ for random search reflects inter-instance heterogeneity (a known property of exponential mixtures). $\Delta\beta$ measures how much each solver homogenizes instance difficulty.

The random search baseline $\beta = 0.87 < 1$ is not anomalous: it is the expected signature of a mixture of exponential distributions with varying success probabilities across instances (the decreasing hazard rate property of mixtures). Each solver's $\Delta\beta$ relative to this baseline measures the degree to which the algorithm *homogenizes* instance difficulty.

Note that c and β are not monotonically related (c: Random > CDCL > WalkSAT; β: WalkSAT > CDCL > Random): they measure different aspects of the solver–structure interaction. Sensitivity c captures how cost *scales* with δ; $\Delta\beta$ captures how much the solver *equalizes* instances of varying difficulty.

5 Discussion

5.1 The sensitivity exponent c

The exponent c in $\mu_c \propto e^{c\delta}$ measures the algorithm's *sensitivity* to the clause density reparameterization δ. The theoretical upper bound is $c = 1$ (random search), since no algorithm can have its cost grow faster with δ than the solution density shrinks.

The observation $c_{\text{WalkSAT}} < c_{\text{CDCL}}$ is instructive. One might expect that a more sophisticated algorithm would have lower c, interpreting small c as "more efficient." The data refute this: CDCL is faster by 10–50× yet *more* sensitive to δ. This is because CDCL's clause learning mechanism *directly engages* with the constraint structure—conflict-driven learning reads and responds to δ, making CDCL's cost track structural complexity more closely. WalkSAT's local search, by contrast, operates with minimal awareness of global structure, resulting in lower sensitivity but higher absolute cost.

Thus c and absolute efficiency ($1/A$ in Eq. 6) constitute two independent axes of algorithmic characterization:

- c (slope): how much does δ amplify the cost?
- $1/A$ (intercept): how fast is the solver at $\delta = 0$?

A complete characterization of a solver's relationship to problem structure requires both.

Connection to proof complexity. The sensitivity exponent c is related to, but distinct from, proof complexity measures. Ben-Sasson and Wigderson [14] showed that resolution proof width lower-bounds proof size, and Pipatsrisawat and Darwiche [15] proved that CDCL with restarts can polynomially simulate general resolution. These results characterize *worst-case* proof length; c instead captures how the *typical* runtime scales with δ, a distributional rather than worst-case measure. The analytical DPLL results of Cocco and Monasson [11] are closer in spirit: their α-dependent exponential rate for DPLL corresponds to a specific value of c in our framework, though the relationship is not exact because their analysis assumes a pure backtracking solver without clause learning.

5.2 The α–n asymmetry

The quantity $\delta = \alpha n|\ln(7/8)|$ enters symmetrically in α and n, but computational cost does not respect this symmetry. Mu and Hoos [7] demonstrated that SLS-based solvers (WalkSAT/SKC, BalancedZ, probSAT) scale *polynomially* with n at the phase transition ($\mu_c \sim n^b$ with $b \approx 3$), while DPLL-based solvers scale exponentially ($\mu_c \sim 1.03^n$).

If the model $\mu_c = A \cdot e^{c\delta}$ held uniformly across both axes, it would predict $\mu_c \propto e^{c\alpha|\ln(7/8)|\cdot n}$ at fixed α—exponential in n for any $c > 0$. For WalkSAT with $c \approx 0.21$ at $\alpha_c \approx 4.27$, this gives $\mu_c \propto 1.13^n$, which is incompatible with the polynomial scaling found by [7].

Direct evidence from n-scaling experiments. Our own WalkSAT experiments at $n = 100$, 200, and 300 confirm this incompatibility (Appendix A). At clause densities where all instances are solved (no survivor bias), the growth *decelerates* with n:

α	γ ($\mu_c \sim n^\gamma$)	Deceleration ratio	Character
3.0	1.07	0.48	nearly linear
3.5	1.69	0.75	polynomial

The deceleration ratio (growth rate n: 200→300 divided by 100→200) equals 1.0 for exponential and < 1 for polynomial growth; both values are clearly sub-exponential. The power-law exponent γ increases with α, consistent with Mu and Hoos's $\gamma \approx 3.2$ at $\alpha_c \approx 4.27$.

Meanwhile, the α-sensitivity exponent c itself shows n-dependence for both solvers when measured at a common clause density range (Appendix A): at $\alpha = 3.5$–4.0, CDCL gives $c = 0.26$, 0.22, 0.24 (range 18%) and WalkSAT gives $c = 0.21$, 0.21, 0.20 (range 7%). Bootstrap CIs confirm that WalkSAT's c decreases significantly from $n = 100$ to $n = 300$ (95% CIs: $[0.18, 0.29]$ and $[0.11, 0.14]$, non-overlapping), consistent with polynomial n-scaling compressing the apparent α-sensitivity at larger n. This n-dependence is a symptom of the product structure $\delta = \alpha \cdot n \cdot |\ln(7/8)|$ being an imperfect coordinate: a model that treats α and n symmetrically cannot fully accommodate the solver-specific n-scaling. For WalkSAT, this is acute: polynomial n-scaling is fundamentally incompatible with the exponential prediction.

Resolution. The sensitivity exponent c characterizes the *α-sensitivity* of computational cost at fixed n, not a universal scaling law. The intercept A in Eq. (6) implicitly depends on n, absorbing the n-scaling. A more complete model takes the form

$$\ln \mu_c(\alpha, n) = c \cdot \alpha\, n\, |\ln(7/8)| + g(n),$$

where $g(n)$ captures the n-scaling and differs qualitatively by solver family:

- **CDCL:** $g(n) \approx -0.075\, n$ (linear), yielding at $\alpha_c \approx 4.27$ an effective growth rate $\mu_c \propto 1.034^n$—consistent with Mu and Hoos's 1.03^n for DPLL. Both directions are exponential, so δ functions as a valid (if imperfect) unified coordinate.

- **WalkSAT:** Polynomial n-scaling ($\mu_c \sim n^{\gamma(\alpha)}$) requires $g(n) \sim \gamma \ln n - c \cdot \alpha \cdot n \cdot |\ln(7/8)|$—a term that must grow as $-\Theta(n)$ to cancel the exponential prediction. The product structure of δ fundamentally breaks down, and c itself becomes n-dependent.

This asymmetry reflects the known distinction between constraint density and search space size: α controls the former while n controls the latter. CDCL—which reads the constraint structure via clause learning—responds exponentially to both; WalkSAT—which operates locally—responds exponentially to α but only polynomially to n. Accordingly, δ is best understood as a reparameterization of α at fixed n, not as a universal coordinate over the (α, n) plane.

5.3 Limitations and future work

Problem size and floor effect. Our results cover $n = 100$–300. At $\alpha \leq 3.0$, the median CDCL runtime (3–8 conflicts) is dominated by solver startup overhead, creating a floor in $\ln \mu_c$ that biases the linear fit downward. The estimated c consequently depends on the clause density range: fits including $\alpha = 3.0$ give $c \approx 0.19$, while fits restricted to $\alpha \geq 3.5$ give $c \approx 0.24$ (consistent with the fine-grained estimate in Appendix D). A cross-n prediction test (Appendix B) achieves factor-of-2 accuracy over three orders of magnitude in μ_c, demonstrating nontrivial predictive power despite the floor effect. Extension to $n \geq 500$—where clustering phenomena become more pronounced [2]—remains untested.

Clause density range. We restrict to $\alpha \leq 4.0$ to avoid survivor bias. The behavior of c near the satisfiability threshold $\alpha_c \approx 4.27$ remains an open question. Fine-grained experiments ($\Delta\alpha = 0.05$) show local slope fluctuations for both solvers, but the signal-to-noise ratio at $\Delta\delta \approx 4$ nats is insufficient to resolve systematic α-dependence.

Cross-domain analogies. The existence–discovery separation and the role of a sensitivity exponent may extend beyond SAT. In large language model (LLM) chain reasoning, externalizing intermediate steps (chain-of-thought) reduces the effective decay rate from $\alpha \approx 0.6$ to $\alpha \approx 0$, paralleling how algorithmic sophistication modifies c in SAT. However, the LLM decay rate measures success probability per chain step, while c measures cost scaling per unit δ; the correspondence is suggestive but not quantitative. More broadly, extending the $\mu_c \propto e^{c\delta}$ model to other domains requires an operational definition of δ (the structural constraint parameter) that is independent of the outcome it predicts—a condition that SAT satisfies via the first moment method but that is nontrivial elsewhere.

6 Conclusion

We have separated the question of solution *existence* from solution *discovery* in random 3-SAT:

$$P(\text{found}) = P(\text{SAT}) \times P(\text{found} \mid \text{SAT}).$$

The first factor is governed by the first moment method with $c = 1$. The second factor—quantified here for the first time—shows that the median computational cost scales as $\mu_c \propto e^{c\delta}$ with $c < 1$ for nontrivial algorithms, and that μ/μ_c serves as an effective normalizing variable for the discovery transition.

The sensitivity exponent c is solver-dependent (CDCL: $\sim$0.24, WalkSAT: $\sim$0.21, Random: 1.0 in the floor-free regime $\alpha \geq 3.5$). The ordering $c_{\text{CDCL}} > c_{\text{WalkSAT}}$ holds at all problem sizes tested ($n = 100$–300), though both solvers show moderate n-dependence (CDCL: range 18%, WalkSAT: range 7% at $\alpha = 3.5$–4.0). For CDCL, δ serves as an effective unified coordinate for both α- and n-scaling; for WalkSAT, δ's product structure does not extend to the n-direction due to polynomial n-scaling [7], and c is best interpreted as an α-sensitivity at fixed n. Across solver

families, c captures a property distinct from efficiency: sophistication *increases* sensitivity to problem structure ($c_{\text{CDCL}} > c_{\text{WalkSAT}}$), even as it decreases absolute cost ($A_{\text{CDCL}} \ll A_{\text{WalkSAT}}$).

The same first-moment exponent δ that governs solution existence also predicts how computational cost scales with clause density—but with a solver-dependent discount factor $c < 1$. Runtime distributions have been extensively studied for algorithm selection and restart design [8, 9, 5], and the first moment method is a standard tool in random CSP theory [1]. Our contribution is the quantitative bridge: the same exponent δ from the first moment method predicts how the *median* of the RTD scales with clause density, with the solver-dependent discount factor c as the key new quantity.

A n-scaling of the sensitivity exponent

n	c (slope)	Intercept a	R^2	Instances
100	0.184	-5.86	0.92	$100/\alpha$
200	0.194	-13.38	0.99	$100/\alpha$
300	0.191	-21.18	0.98	$200/\alpha$

Table 4: CDCL: Sensitivity exponent c and intercept a (from $\ln \mu_c = c\delta + a$) across problem sizes. Note: the α-ranges differ by n ($n = 100{,}200$: $\alpha = 3.0$–4.27; $n = 300$: $\alpha = 3.0$–4.0), and $\alpha = 3.0$ falls in the floor regime (median conflicts 3–8, Section 5). The intercept decreases approximately linearly with n: $\Delta a/\Delta n \approx -0.075$.

CDCL. The intercept $a(n)$ decreases linearly with n, meaning the full model is

$$\ln \mu_c(\alpha, n) \approx c \cdot \alpha\, n\, |\ln(7/8)| + g(n), \quad g(n) \approx -0.075\, n + 1.6.$$

At fixed α, the effective exponential growth rate in n is $c \cdot \alpha \cdot |\ln(7/8)| - 0.075$. At $\alpha = \alpha_c \approx 4.27$:

$$\text{rate} = 0.19 \times 4.27 \times 0.134 - 0.075 \approx 0.033,$$

predicting $\mu_c \propto 1.034^n$. (Here $c = 0.19$ and $g(n)$ are paired values from the same full-range fit; using floor-free $c \approx 0.24$ would require a correspondingly steeper $g(n)$, yielding a similar net rate.) This is consistent with Mu and Hoos's [7] finding that DPLL-based solvers scale as $\sim 1.03^n$. The negative intercept $g(n)$ thus absorbs most of the n-dependence, reducing the naive prediction from 1.11^n (slope alone) to 1.034^n (slope + intercept).

n	c (slope)	Intercept a	R^2	Instances
100	0.172	-2.16	0.98	$50/\alpha$
200	0.144	-6.18	0.95	$50/\alpha$
300	0.143	-11.62	0.95	$50/\alpha$

Table 5: WalkSAT: Sensitivity exponent c and intercept a across problem sizes (full-range fit including $\alpha = 3.0$). The slope c decreases with n (range 18.9%, mean $\bar{c} = 0.153$). At a common clause density range ($\alpha = 3.5$–4.0, local slope), the n-dependence is smaller: $c = 0.21, 0.21, 0.20$ (range 7%).

WalkSAT. At clause densities with no survivor bias ($\alpha = 3.0, 3.5$; solve rate $= 1.0$ at all n), the median runtime grows as a power law in n:

The deceleration ratio is clearly below 1 at both clause densities, ruling out exponential n-scaling. The increasing γ with α is consistent with Mu and Hoos's [7] finding of $\gamma \approx 3.2$ at

α	$\mu_c(100)$	$\mu_c(200)$	$\mu_c(300)$	γ	Decel. ratio
3.0	124	274	402	1.07	0.48
3.5	288	836	1,846	1.69	0.75

Table 6: WalkSAT median flips at bias-free clause densities. The power-law exponent γ (from $\mu_c \sim n^\gamma$) and the deceleration ratio (= growth 200→300 / growth 100→200; equals 1.0 for exponential, < 1 for polynomial) both confirm sub-exponential n-scaling.

$\alpha_c \approx 4.27$. Higher clause densities ($\alpha = 3.86, 4.0$) show accelerating growth at $n = 300$, but with solve rates of 0.84 and 0.60 respectively, survivor bias makes these estimates unreliable.

Together, the CDCL and WalkSAT data confirm that $\delta = \alpha \cdot n \cdot |\ln(7/8)|$ serves as a valid unified coordinate only for solvers whose n-scaling is also exponential (CDCL). For polynomial-n solvers (WalkSAT), c remains a useful measure of α-sensitivity at fixed n, but the product structure of δ does not extend to the n-direction.

B Cross-n prediction test

To test whether the model $\ln \mu_c = c\delta + g(n)$ has *predictive* (not merely descriptive) power, we train a pooled regression $\ln \mu_c = c\,\delta + b\,n + a_0$ on data at $n = 100$ and $n = 200$ (common $\alpha = 3.0$, 3.5, 4.0; 6 training points per solver) and predict μ_c at $n = 300$.

α	μ_c^{pred}	μ_c^{actual}	Ratio	$\lvert\Delta \ln \mu_c\rvert$
CDCL (conflicts)				
3.0	6	8	0.70×	0.36
3.5	456	158	2.89×	1.06
4.0	37 323	21 170	1.76×	0.57
WalkSAT (flips)				
3.0	295	402	0.73×	0.31
3.5	5 871	1 846	3.18×	1.16
4.0	116 945	102 881	1.14×	0.13

Table 7: Cross-n prediction: pooled regression on $n = 100{+}200$ predicts μ_c at $n = 300$. Both solvers achieve RMSE ≈ 0.7 in log space (factor $\sim 2\times$) over a dynamic range of ~ 8 nats ($\sim 2600\times$). The largest errors occur at $\alpha = 3.5$, where residual floor effects from the training data (CDCL: 19 conflicts at $n = 100$) inflate the predicted slope.

The RMSE in log space is 0.73 for CDCL and 0.70 for WalkSAT (both corresponding to a factor of $\sim 2\times$). This factor-of-2 accuracy, achieved over a range where μ_c spans three orders of magnitude, demonstrates nontrivial predictive power of the $\ln \mu_c = c\delta + b\,n + a_0$ model. That both solvers achieve comparable prediction accuracy suggests that the dominant source of error is the n-dependence of the floor effect, not the solver-specific scaling.

C Model comparison

We compare three functional forms for the transition curve $S_{\text{solve}}(\mu/\mu_c)$: sigmoid, exponential CDF, and Weibull CDF.

Weibull shape parameters: CDCL $\beta = 0.93$, WalkSAT $\beta = 1.10$. The 15% difference in Weibull β mirrors the 14% difference in sigmoid k, confirming that the transition shape difference is model-independent.

Solver	Best model	2nd (ΔAIC)	3rd (ΔAIC)
CDCL	Weibull (0)	Exp CDF (+2.8)	Sigmoid (+13.7)
WalkSAT	Sigmoid (0)	Weibull (+13.2)	Exp CDF (+13.2)

Table 8: AIC model comparison. The best-fit model is solver-dependent: Weibull for CDCL, sigmoid for WalkSAT.

D Clause density profile

Figure 3 shows the median CDCL runtime as a function of clause density α at $n = 200$, with fine-grained spacing $\Delta\alpha = 0.1$ over the range $\alpha = 3.5$–4.2 (instances with $\alpha > 4.2$ are excluded due to survivor bias). The left panel reveals roughly two orders of magnitude growth in μ_c across this range. The right panel shows $\ln \mu_c$ versus δ, decomposed into two segments at the dynamic threshold $\alpha_d \approx 3.86$. The local slope is steeper above α_d ($c = 0.26$) than below ($c = 0.18$), suggesting that the onset of clustering in the solution space [2] accelerates the growth of computational cost. The global linear fit ($c = 0.24$) captures the overall trend, indicating that the kink is a second-order correction. Note that $c = 0.24$ here (8 α-values, $n = 200$) differs from $c = 0.19$ in Section 4.1 (4 α-values, $n = 200$); the wider α-range shifts the estimate upward.

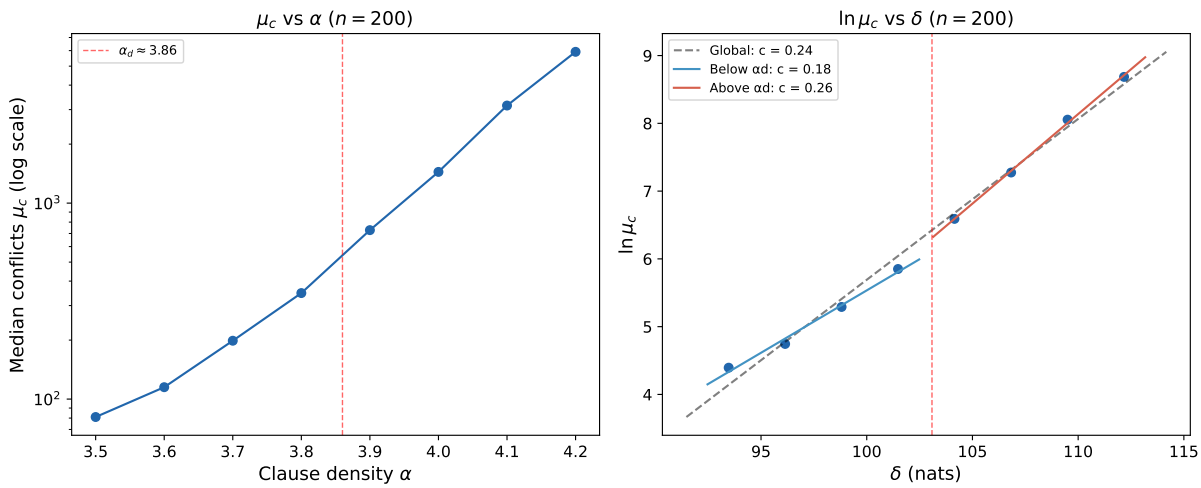

Figure 3: Median CDCL runtime versus clause density ($n = 200$, 500 instances per α, $\alpha = 3.5$–4.2). Left: μ_c on log scale. Right: $\ln \mu_c$ vs δ with piecewise linear fits ($c = 0.18$ below α_d, $c = 0.26$ above). The vertical dashed line marks $\alpha_d \approx 3.86$.

E Computational budget metrics

Metric	c (CDCL)	R^2
Conflicts	0.194	0.991
Propagations	0.082	0.757
Decisions	0.040	0.488

Table 9: Sensitivity exponent for different budget metrics ($n = 200$). Conflicts yield the cleanest multiplicative separation, consistent with conflicts representing the fundamental state transitions of the CDCL algorithm.

References

[1] M. Mézard, A. Montanari, *Information, Physics, and Computation*, Oxford University Press, 2009.

[2] D. Achlioptas, A. Coja-Oghlan, "Algorithmic barriers from phase transitions," *Proc. FOCS*, pp. 793–802, 2008.

[3] J. Ding, A. Sly, N. Sun, "Proof of the satisfiability conjecture for large k," *Proc. STOC*, pp. 59–68, 2015.

[4] T. Schöning, "A probabilistic algorithm for k-SAT and constraint satisfaction problems," *Proc. FOCS*, pp. 410–414, 1999.

[5] C. P. Gomes, B. Selman, N. Crato, H. Kautz, "Heavy-tailed phenomena in satisfiability and constraint satisfaction problems," *J. Automated Reasoning*, 24(1/2):67–100, 2000.

[6] C. Coarfa, D. D. Demopoulos, A. San Miguel Aguirre, D. Subramanian, M. Y. Vardi, "Random 3-SAT: The plot thickens," *Constraints*, 8(3):243–261, 2003.

[7] Z. Mu, H. H. Hoos, "On the empirical time complexity of random 3-SAT at the phase transition," *Proc. IJCAI*, pp. 367–373, 2015.

[8] H. H. Hoos, T. Stützle, "Towards a characterisation of the behaviour of stochastic local search algorithms for SAT," *Artificial Intelligence*, 112(1–2):213–232, 1999.

[9] D. Frost, I. Rish, L. Vila, "Summarizing CSP hardness with continuous probability distributions," *Proc. AAAI*, pp. 327–333, 1997.

[10] J. Aczél, *Lectures on Functional Equations and Their Applications*. Academic Press, 1966.

[11] S. Cocco, R. Monasson, "Trajectories in phase diagrams, growth processes, and computational complexity: How search algorithms solve the 3-satisfiability problem," *Physical Review Letters*, 86(8):1654–1657, 2001.

[12] J. P. Marques-Silva, K. A. Sakallah, "GRASP: A search procedure for propositional satisfiability," *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[13] B. Selman, H. A. Kautz, B. Cohen, "Noise strategies for improving local search," *Proc. AAAI*, pp. 337–343, 1994.

[14] E. Ben-Sasson, A. Wigderson, "Short proofs are narrow—resolution made simple," *Journal of the ACM*, 48(2):149–169, 2001.

[15] K. Pipatsrisawat, A. Darwiche, "On the power of clause-learning SAT solvers as resolution engines," *Artificial Intelligence*, 175(2):512–525, 2011.